



slingshot college
(इस्लिङ्गटन कलेज)

Module Code & Module Title

CC6010NI Digital Investigation and E-Discovery

Assessment Weightage & Type

50% Individual Coursework

Year and Semester

2025 -26 Autumn Semester

Student Name: Niyati Pachhai

London Met ID: 23047408

College ID: 230110

Assignment Due Date: January 21st, Wednesday

Assignment Submission Date: January 20th, Tuesday

Word Count (Where Required): 5120

I confirm that I understand my coursework needs to be submitted online via MST under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Abstract

With the growing trend of relying on remote access and cloud computing, authentication logs have emerged as a valuable source of digital evidence that helps in revealing security breaches. Conventional rule-based security solutions prove less effective in pinpointing sophisticated and low and slow attacks, including those that are indistinguishable from usual legitimate user activities. This research work proposes an anomaly-based method of SSH authentication log analysis that involves machine learning security logic. An Isolation Forest algorithm is deployed on features like login time, failed login attempts, and novelty of source IP addresses in SSH authentication log files. To enhance forensic validity, domain-level security rules are applied to give priority to high-risk conditions that involve repeated failed login attempts in odd hours on new machines and IP addresses. The system yields a numerical risk value and assigns the risk level to events, helping to hasten their assessment, and the outcome illustrates that a hybrid of machine learning and security logic improves accuracy in anomaly detection and ensures accountability and transparency in digital forensic analysis.

Acknowledgement

I would also like to take this opportunity to thank my tutor, Mr. Abhishek Bhatta sir, for all the assistance that I received throughout the development stage of this project. His inputs regarding digital forensics methods as well as cybersecurity theory were crucial in developing the analysis framework used in my project. I would also like to take this opportunity to thank the module leader, Mr. Anuj Shilpakar sir, for devising the module material that formed a sound theoretical and practical basis for this project.

In addition, I appreciate the utilization of publicly available authentication log format files and security research resources in the process of developing this anomaly detection system. Finally, I appreciate my colleagues and institution for cultivating an environment conducive to academia and innovation in critical thinking and applying theoretical constructs to real-world digital forensics.

Table of Contents

1.	Introduction.....	1
1.1	Intrusion Detection and Security Monitoring	2
1.2	Machine Learning.....	2
1.3	Aim and Objectives	3
1.3.1.	Aim	3
1.3.2.	Objectives.....	3
2.	Background	4
2.1.	Case Study 1: Equifax data breach (2017)	4
2.1.1.	The Attack Mechanism	4
2.1.2.	Discovery and Consequences	5
2.1.3.	Summary	6
2.2.	Case Study 2: Capital One Data Breach.....	7
2.2.1.	The Attack Mechanism	7
2.2.2.	Discovery and Consequences	8
2.2.3.	Summary	9
2.3.	Learning from Case Studies.....	10
2.4.	Personal Perspective	11
2.5.	Tool, Technology, and Approach	11
3.	Methods, Opportunities and Motive	13
3.1.	Methods	13
3.2.	General Weaknesses/Opportunities	15
3.3.	Motive.....	15
4.	Critical Forensic Analysis.....	16
4.1.	Forensic Process Diagram.....	16
4.2.	Attack Demo.....	17
4.3.	Analysis of Login Risk	25
5.	Result and Evaluation.....	27

6.	Conclusion	31
7.	References	32

Table of Figures

Figure 1: Vulnerabilities being exploited (breachsense, 2024)	5
Figure 2: Exploitation of a server-side request forgery (SSRF (Shaharyar Khan, 2022)).	7
Figure 3: Process of a server-side request forgery (SSRF) (Thomas E. Jeffry, 2020)	8
Figure 4: SSRF-based attack flow showing credential theft and data access in the Capital One breach (Insider Lab, 2025)	14
Figure 5: Forensic Process Diagram	16
Figure 6: Loading login event data	17
Figure 7: Importing required libraries	17
Figure 8: Risk Score Classification	18
Figure 9: Loading Authentication Logs	18
Figure 10: Detecting required login feature	19
Figure 11 Training the Isolation Forest model	19
Figure 12: Calculating anomaly scores	20
Figure 13: Rule based override	20
Figure 14: Importing libraries	21
Figure 15: Loading anomaly detection results	21
Figure 16: Plotting normal and anomalous login events	22
Figure 17: Labels and title	23
Figure 18: Failed login identification	23
Figure 19: Identifying Security Events	24
Figure 20: IP Address extraction	24
Figure 21: Sample login records	25
Figure 22: Distribution of normal vs anomalous login events	27
Figure 23: Average risk score	28
Figure 24: Anomaly detection based on login behaviour	29

1. Introduction

Today's organizations are bombarded by an unending stream of cyber-attacks. Hence, it is necessary to have real-time alerts for security events. Generally, if a prolonged time passes without detection of intrusion into a computer network/cyber space, it may lead to higher damage (Scarfone, 2025). Even now, intrusion detection systems are the security tools which are used to protect data and monitor events (e.g. logins, network traffic) (Nist, 2025). Adding to that are traditional intrusion detection techniques like signature-based IDS because it depends on known patterns of attacks. On the flip side, it fails to identify attacks when they are completely new to it. Other techniques include anomaly-based IDS because it highlights deviations from what is expected. Its advantage is to be able to point out attacks when they were previously not knowledgeable about them. Conversely, it may possibly give higher "false positives" (tsapps.nist.gov, 2025).

Machine learning (ML) has also appeared to be a beneficial method in intrusion detections, which enables computers to train complicated patterns of normal and intrusion behaviours (Andrea Pinto 1, 2022). In recent years, many scholars employed numerous methods of ML in recently emerging fields of cybersecurity, including traditional machine learning approaches, including one-class SVMs and Clustering methods, as well as Deep Neural Networks, including Autoencoders, and LSTMs. For instance, researchers employed a hybrid strategy of One-class SVM & Isolation Forest to identify network intrusion in IoT networks, which indicated that lightweight unsupervised machine learning methods are efficient at discovering abnormalities. More recently, neural networks are also employed for log-based anomaly detections, as well as Hybrid Intrusion Detection Systems combining signature & anomaly-based detections. In all these cases, a prominent conclusion was reached, which indicated that the usage of anomaly-based IDS systems can considerably improve the detections of sophisticated attacks.

1.1 Intrusion Detection and Security Monitoring

An Intrusion Detection System is a type of security control that can monitor activities automatically in either computer or network setups to detect possible cues of intrusion attempts (Scarfone, 2025). There are several types and models of IDS. These range from network-based IDS to host-based IDS. Network-based IDS examines packet flow within the network as can monitor network activities. Host-based IDS is based on the setup and runs on each computer to monitor activities such as login attempts or accessing files. Essentially, the primary role of IDS is to raise alerts as soon as intrusion attempts are detected. Incident response is typically related to IDS (Douligeris, 2007).

Conventionally, there have been two types of approaches to IDS systems, namely signature-based detection systems and anomaly-based systems. Signature-based detection systems keep a knowledge base consisting of threat patterns. These systems generate an alarm when it encounters a particular threat with a known bad signature (Douligeris, 2007). This approach is accurate regarding already-known threats but unable to cope with novel threats for which no signature has been created yet. Anomaly-based systems, on the other hand, generate a baseline of systems' usual characteristics based on machine learning techniques applied to the input patterns. An example would be where normally there would be no more than 100 queries per minutes from the application server to the database, but thousands are observed (Douligeris, 2007).

1.2 Machine Learning

Machine learning enhances anomaly-based IDS by automating the baseline learning and improving pattern recognition beyond simple statistical thresholds. ML-based IDS can utilize a range of algorithms: supervised learning models-e.g., decision trees, SVMs, and deep neural networks-trained on labelled attack vs. normal data to classify traffic, and unsupervised models-e.g., clustering, isolation forests, autoencoders-that learn normal patterns

without labelled examples and detect outliers (Diphoko, 2024). Once deployed, an ML-IDS will analyse incoming data in real time and produce alerts for anything deemed sufficiently anomalous or similar learned attack patterns.

The ML pipeline typically involves collecting large volumes of security data, such as network flows and system logs, extracting relevant features (e.g., connection duration, request rates, user behaviour metrics), and continuously training models to distinguish benign from malicious behaviour (Diphoko, 2024).. Early deployments as well as studies have shown better detection strength, with examples such as one open-source project called “IDS-ML,” which illustrated well-optimized ML models successfully detecting various types of attacks within benchmark datasets, thereby indicating real-world network potentials (Li Yang, 2022).

1.3 Aim and Objectives

1.3.1. Aim

The aim of this project is the detection of anomalous login through the analysis of system login logs based on machine learning approach in order to spot suspicious or malicious login activity.

1.3.2. Objectives

- To store data of Authentication Log entries for extracting significant behavioural features from the login records
- To apply an anomaly-based machine learning model to spot suspicious or malicious login activity.
- To produce interpretable risk scores and classification.
- To visualize and interpret the detected anomalies.

2. Background

2.1. Case Study 1: Equifax data breach (2017)

Equifax, known as a leading US credit bureau, has disclosed a massive data breach which the company claims to have exposed sensitive personal and financial data for 143 million American consumers due to which their names and addresses and social security numbers, birthdates none of that can get reissued (DeBrusk, 2022). There was no report indicating a vulnerability to the Apache Struts problem. Therefore, these systems were unpatched and unprotected until July 29, 2017, (Amishav, 2024).

During the intrusion, the attackers discovered a file with unencrypted login credentials, which allowed them to bypass the security of the ACIS environment and infiltrate 48 additional, unrelated databases. The attackers executed approximately 9,000 queries across these systems, identifying unprotected personally identifiable information (PII) on 265 separate occasions. This sensitive data was subsequently exfiltrated from Equifax's servers without the company's knowledge (Majority Staff Report, 2018).

2.1.1. The Attack Mechanism

In the company's first announcement, Equifax said that attackers had penetrated its systems from mid-May through July 2017. The vulnerability that allowed attackers to enter Equifax's systems and make off with the data was known as Apache Struts CVE-2017-5638, which allows attackers to execute arbitrary commands from a remote location (Thomas, 2019). On March 7, 2017, a major security flaw in Apache Struts was revealed. Equifax relied on this software for various applications running on older operating systems. The next day, the Department of Homeland Security notified Equifax of the danger. On March 9, Equifax's Global Threat and Vulnerability Management (GTVM) team sent an email alert to more than 400 staff members, giving them 48 hours to patch any systems using Apache Struts. The GTVM team followed up with a meeting regarding the vulnerability on March 16 (Majority Staff Report, 2018).

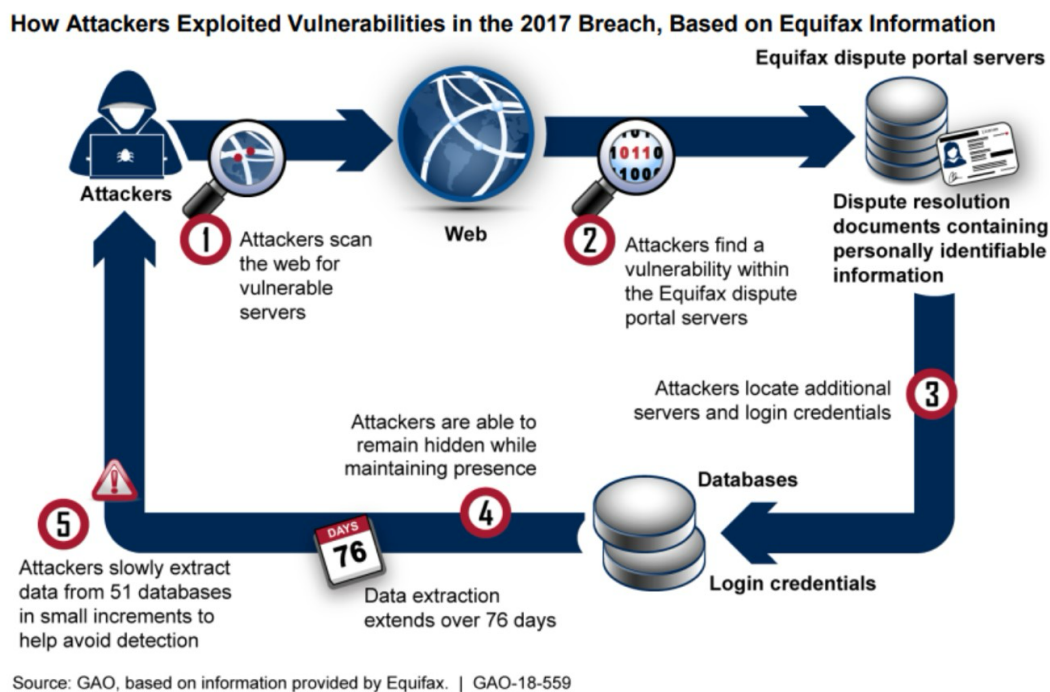


Figure 1: Vulnerabilities being exploited (breachsense, 2024)

Despite these efforts, Equifax failed to completely update its infrastructure. Specifically, its custom-built, web-accessible Automated Consumer Interview System (ACIS) a portal used for consumer disputes remained unpatched.

2.1.2. Discovery and Consequences

Equifax did have network monitoring tools to decrypt, analyse and re-encrypt the data, but the monitoring tool didn't work because an expired certificate for the company's TLS traffic verification was still present. Equifax haven't renewed their cert, and traffic was not being decrypted. They had no idea what data was going out of their network. On May 13, 2017, the intruders moved laterally to other servers in the network because of no appropriate network segmentation. Once there, the attackers located plaintext passwords, providing them with even more access to servers (Amishav, 2024)

The security department realized that there was some malicious activity on their network that was observable from their side. They took their application offline and analysed three days and later hired an external cybersecurity firm to conduct a forensic investigation. The Initial reports showed that several files had been compromised. This went on to announcements of the personal information of 145 million Americans and 8,000 Canadians having been compromised, and 693,000 British citizens' data had been affected because of a data breach (Thomas, 2019).

2.1.3. Summary

The Equifax breach illustrates how vulnerable exposure coupled with ineffective behavioural analysis can lead to extended unauthorized access. An existing vulnerability in Apache Struts was used to breach Equifax's web environment, and continued access was maintained for a span of weeks. From a digital forensics point of view, the attack was a function not only of missing patches, but more critically, a lack of detection for anomalous application and data accesses. The web server logs, application error logs, and SQL server access logs would have indicated statistically anomalous patterns of access, such as an anomalous request format, a high rate of errors, and continuous high levels of access to the data files. However, such patterns were not appropriately integrated, permitting substantial amounts of data to be harvested.

Key Findings

- Anomalies in web traffic alerts that rule-based systems fail to leverage sufficiently are good leading indicators.
- Employees must be trained to identify unusual sign and suspicious behaviour.

2.2. Case Study 2: Capital One Data Breach

The Capital One Financial Corporation is one of the world's largest banks. It mostly offers credit cards, auto loans, banking, and savings services. Capital One is one of the best banks in the United States. It was founded in 1994 and is based in McLean, Virginia. It is known for its investments in technology and marketing strategies (US. Attorney, 2022). Capital one is notable for their early and aggressive adoptions in cloud computing, migrating a substantial proportion of their infrastructure to the AWS (Amazon Web Services) for better innovation and scalability.

According to Capital One, the event took place on March 22, 2019, after the hacker used the network vulnerabilities that arose due to the misuse of the web application firewall. As a result, the hacker used the process to conduct a Server-Side Request Forgery (SSRF) attack, which fooled the app server into allowing access to the AWS environment (CapitalOne, 2022).

2.2.1. The Attack Mechanism

As stated by Capital One, the incident occurred on March 22, 2019, because of a hacker exploiting their network via a web application firewall misconfiguration". Thus, the process implemented by the hacker allowed him to successfully execute a Server-Side Request Forgery (SSRF) attack

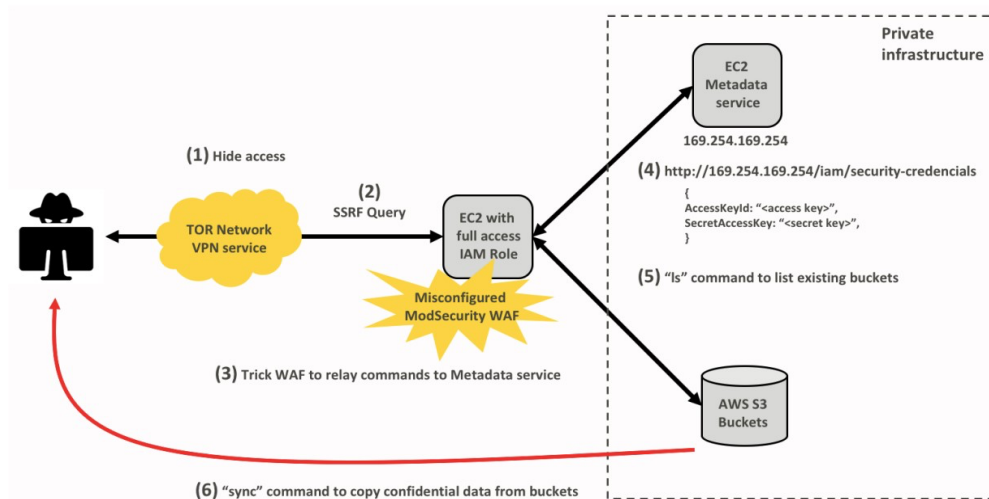


Figure 2: Exploitation of a server-side request forgery (SSRF) (Shaharyar Khan, 2022)

against infrastructure. As a result, they were able to obtain information necessary for accessing data contained within their Capital One Cloud Storage (Shaharyar Khan, 2022).

2.2.2. Discovery and Consequences

As per the FBI report, the Capital One bank was notified of the breach of their data on July 17, 2019. The bank had received an email stating that some of their data was available on GitHub. The Github page, belonging to "Net crave," was associated directly with Thompson through her resume. Furthermore, Thompson also had an online Meetup maintained using the pseudonym "erratic," through which she urged other members to join "Netcrave Communications" on Slack (Thomas E. Jeffry, 2020).

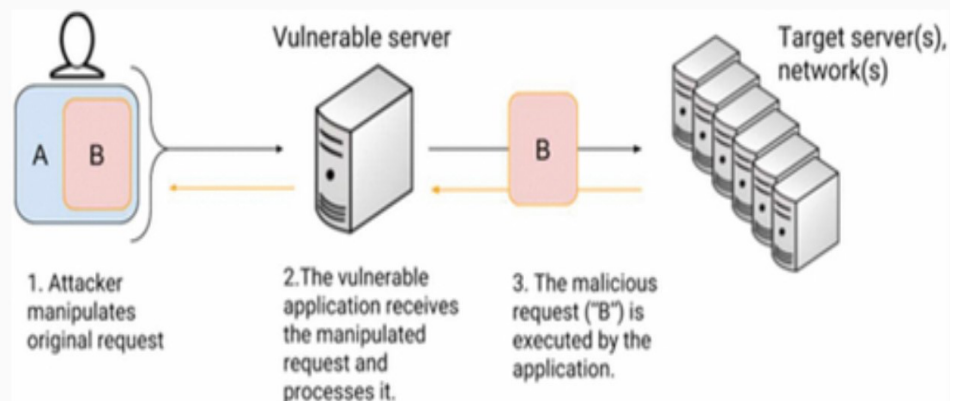


Figure 3: Process of a server-side request forgery (SSRF) (Thomas E. Jeffry, 2020)

With the account credentials, the vulnerability was able to make AWS CLI requests for the list and copy operation for more than 700 customer data storage buckets for the Contracts with Capital One. The hacker took advantage of the vulnerability to obtain information that helped them access the data contained in the capital One's cloud storage, abbreviated as the S3 bucket. The FBI arrested Paige A. Thompson, who was alleged to have accessed about 30GB of data (US. Attorney, 2022).

2.2.3. Summary

The Capital One data breach shows a cloud-centric attack scenario where incorrect configuration and inadequate behavioural monitoring allowed unauthorized data access is exemplified by the Capital One data breach. The attacker gained access to private information kept in object storage services by taking advantage of cloud infrastructure features and a misconfigured web application firewall.

Using those valid credentials, she was able to execute thousands of valid API calls, pulling personal data on more than 100 million customers. It was cloud traffic that looked normal; therefore, traditional Capital One defences did not immediately flag it no signature was triggered, and the anomalous access patterns were not identified as unusually large data retrievals by a single user role. The breach went undetected internally for about four months. Both cases really point out that the timeline from intrusion to detection stretched weeks or months, mostly due to reliance on static rules and known threat signatures. This breach is noteworthy from a forensic perspective because all the attacker's actions including API calls, resource access, and data retrieval operations were documented in cloud audit logs.

2.3. Learning from Case Studies

High-profile breaches such as Equifax in 2017 and Capital One in 2019 demonstrate attack patterns, long escape times before being discovered, undetected, and vulnerabilities that traditional security had not detected. In Equifax, they used an Apache Struts web application vulnerability that had not been patched (CVE-2017-5638) to gain access starting mid-May 2017 (Majority Staff Report, 2018). “The Equifax breach was ‘entirely preventable,’ according to a House oversight investigation. It pointed to a failure to update monitoring certificates that had expired. Equifax could have avoided both security mistakes.” A similar pattern of "authorized" yet abnormal activity was observed in the Capital One incident in March 2019. A hacker (Paige Thompson) utilized a Server-Side Request Forgery (SSRF) weakness to exploit a poorly configured Web Application Firewall (WAF) and obtain AWS credentials. The access allowed her to access sensitive data in cloud storage (S3 buckets) (Shaharyar Khan, 2022)..

In both cases, weaknesses were exploited by attackers, known about those weaknesses, then using properly issued credentials or standard communications protocols to access data. Conventional security measures, designed to detect attacks using known malware or key signatures, failed to detect odd patterns of behaviour: (an unusual spike of database activity late at night in Equifax, or an unusual series of cloud communication API interactions in Capital One) (Nick Marinos, 2018).

The key technical issues were, respectively, poor patch management, lack of sufficient network segregation (making it possible for attackers who undermined a web portal to access critical databases), as well as insufficient real-time monitoring of usage patterns (US. Attorney, 2022). Both cases illustrate a fundamental benefit of anomaly detection: while properly functioning within expected patterns, unusual activity which does not fit expected patterns of usage can still occur. Equifax’s initial failure, as reported by GAO, was that attacks exploited weaknesses of "identification, detection,

segregation of access to databases, and data governance," or, in simpler terms, detection of unusual patterns, which would slow or otherwise limit the attackers' exploitation of those patterns. An analytics solution with machine learning techniques would alert much earlier to unusual patterns of access attempts (Nick Marinos, 2018).

2.4. Personal Perspective

From my perspective, it is difficult to underestimate the importance of cybersecurity or anomaly detection in today's world. Breaches such as Equifax and Capital One shook millions of people to their cores. These breaches are a symptom of attackers outpacing traditional methods. Cyber attackers are always adapting their methods, and we need to evolve at an exponential rate. According to recent reports, it currently takes an organization an average of 194 days in 2024 to realize breaches once a hack has occurred (Sobers, 2025). This is, of course, an unacceptable timeframe for a hacker to operate within a network. It is necessary to think about how AI machine learning can sift through millions of log entries to spot a specific entry regarding a hacker login.

2.5. Tool, Technology, and Approach

This project uses a variety of software tools and libraries to implement the real-time anomaly detection pipeline. The development and experimentation were performed in the Python environment, which is reflective of Python's status as a dominant language for both data analysis and cybersecurity automation because of its very rich ecosystem of libraries. Below, we outline some of the tools used and their respective roles:

- **Python 3.9:** All scripts, data generation utilities, and analysis components were implemented in Python. Python's simplicity and modularity make it well suited for rapid prototyping of analytical pipelines.

- **Jupyter Notebook:** Jupyter Notebook was used as the primary development and experimentation environment. Its interactive execution model allowed incremental testing of data generation logic, preprocessing steps, and model behavior.
- **Data Handling Libraries (Pandas and NumPy):** The panda's library was used extensively for data manipulation, including loading synthetic login data, cleaning records, engineering features, and maintaining per-user historical state (e.g., identifying whether an IP or device was new).
- **Machine Learning Library – scikit-learn:** Scikit-learn provided the core machine learning functionality through its implementation of Isolation Forest.
- **Visualization Libraries (Matplotlib):** In order to analyse and present results, Matplotlib was used for plotting anomaly score distributions, timelines of login activity, and risk-level trends.
- **Synthetic Data Generator:** Because real authentication datasets are rarely publicly available due to privacy and security concerns, a custom synthetic data generator was developed in Python.
- **VS Code:** Visual Studio Code acted as a development IDE to develop scripts using Python. VS Code was helpful in terms of extensions available for Python and Jupyter notebooks. Development of an anomaly detection system based on the login data.

These assets were combined to set up an experiment to perform simulations related to the login process, develop the model for the detection of anomalies, and plot the results. Even the integration of various Python libraries facilitated the development process of the detection model, and using the well-documented algorithm provided authenticity to the results since it's in the scikit-learn library implementation.

3. Methods, Opportunities and Motive

3.1. Methods

Anomaly-Based Attack Methods: Attackers have increasingly turned to anomaly-based attacks as a means of avoiding traditional methods. Rather than leveraging known malware signatures or exploiting known vulnerabilities in systems or credentials, attackers choose to use guaranteed or known systems/credentials abnormally. This includes scenarios like unusual logins at non-working hours (e.g. during weekends), logins from new or foreign IP addresses/devices, and unusual querying or data uploads not typical of an account (censinet.com, 2025). For instance, an anomaly-based detection software will alert on unusual logins (e.g. an employee ID suddenly accessing from an unknown region or device) and unusual querying of data (e.g. an unusual spike in database and/or file downloads exceeding an employee ID's normal expected rate of download). An Anomaly-Based IDS or User and Entity Behaviour Analytics (UEBA) module would point out these occurrences of unusual or deviating user and entity behavioural patterns from normal norms. This would alert administrators of unusual occurrences like an employee ID logging in and attempting to download an abnormally large number of records at 3 AM (Insider Lab, 2025).

- **Equifax Data Breach**-The attackers used an Apache Struts vulnerability that had not been fixed to enter Equifax's network. Once in, they worked quietly for more than two months, querying databases and stealing valuable data (personal records). They covered their tracks by encrypting their communications. If an IDS had detected this attack activity, it would have shown up as unusual, encrypted transmissions and database requests from a trusted server (hashesout, 2018).
- **Capital One** - In this attack, a former AWS engineer attacked Capital One using a cunning SSRF attack through a misconfigured web application firewall to steal credentials from the AWS metadata service. With compromised cloud

access keys, she extracted and downloaded customer data from an Amazon S3 storage bucket (Insider Lab, 2025).

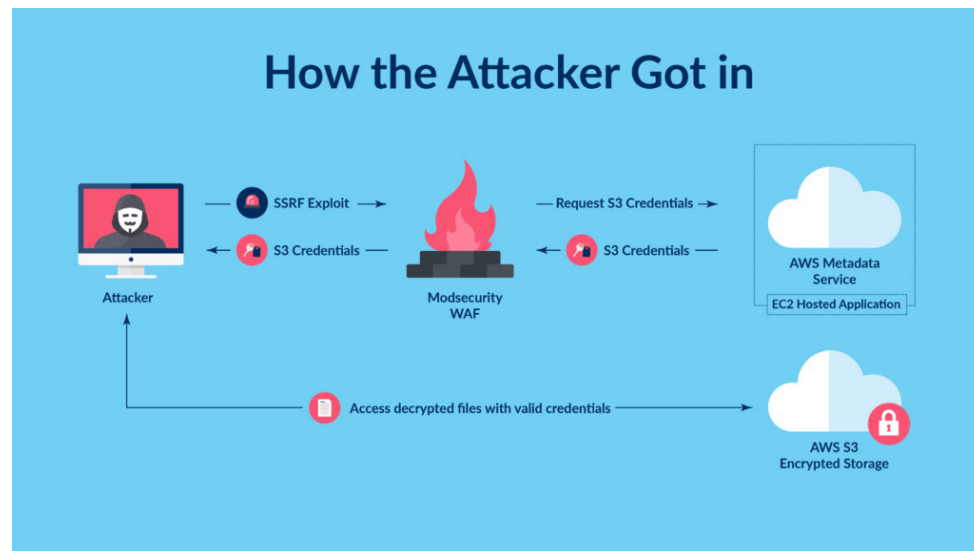


Figure 4: SSRF-based attack flow showing credential theft and data access in the Capital One breach (Insider Lab, 2025)

- **SolarWinds Orion (2020)-** This was a supply-chain attack wherein adversaries implanted a trojanized update into the SolarWinds Orion IT monitoring software, which eventually was pushed to many organizations. The malware, SUNBURST, was designed to mimic normal Orion network traffic using techniques to resemble legitimate communications. The Solar Winds case thus underlines the fact that even extremely sophisticated attacks leave behind behavioural marks-such as anomalous authentications or new network destinations-that may be picked up by an ML-driven anomaly detector even when malware tries to blend in.

Overall, anomaly-based attacks "hide in plain sight" by utilizing valid credentials or software but in ways that deviate from normal patterns. A classic IDS may not flag these if no known signature is triggered, whereas an anomaly-focused system would raise alerts on the bizarre timing, origin, or scale of the activities.

3.2. General Weaknesses/Opportunities:

Research uncovers that most organizations do not successfully identify breaches in a timely manner, which affords attackers a considerable amount of dwell time. This was revealed in the global IBM report in 2023, with only 33% of breaches detected in-house by the organization, and the rest detected by a third party, including the attackers after the fact. Breaches were unknown for an average of over 200 days in 2023, which speaks for itself in the length abnormal activity can go unnoticed before an internal alert is triggered (Alder, 2023). The problem, in most instances, traces back to a lack of proper monitoring against the baseline. Without the constant investigation of logins from unusual geographical locations, out-of-band time, and unusual resource utilization, attackers are given the gift of time to act on the shortcomings. Clearly, any unchecked endpoint, expired control, and investigated anomaly is an "attack window." Modern attackers deliberately hunt for these conditions, and it may be an open RDP port, an expired certificate, and an inactive account with unchecked activity (hashesout, 2018).

3.3. Motive

The end goal of anomaly types of breaches is always to get what they need, whether it is data, funds, or advantage, through methods of escaping the norms. Case studies of actual breaches seem to point out the fact that many were simply not inevitable but rather exploits of weaknesses and a lack of oversight:

The anomalies enable attackers to effectively be among the normal processes of network activity, so they can go undetected to fully gain their goal. The main reason behind these attacks may be profit, cyber espionage, or resale of access on the dark marketplace, and these attacks are less likely to be detected because they operate on anomaly methods, rather than methods commonly known

4. Critical Forensic Analysis

4.1. Forensic Process Diagram

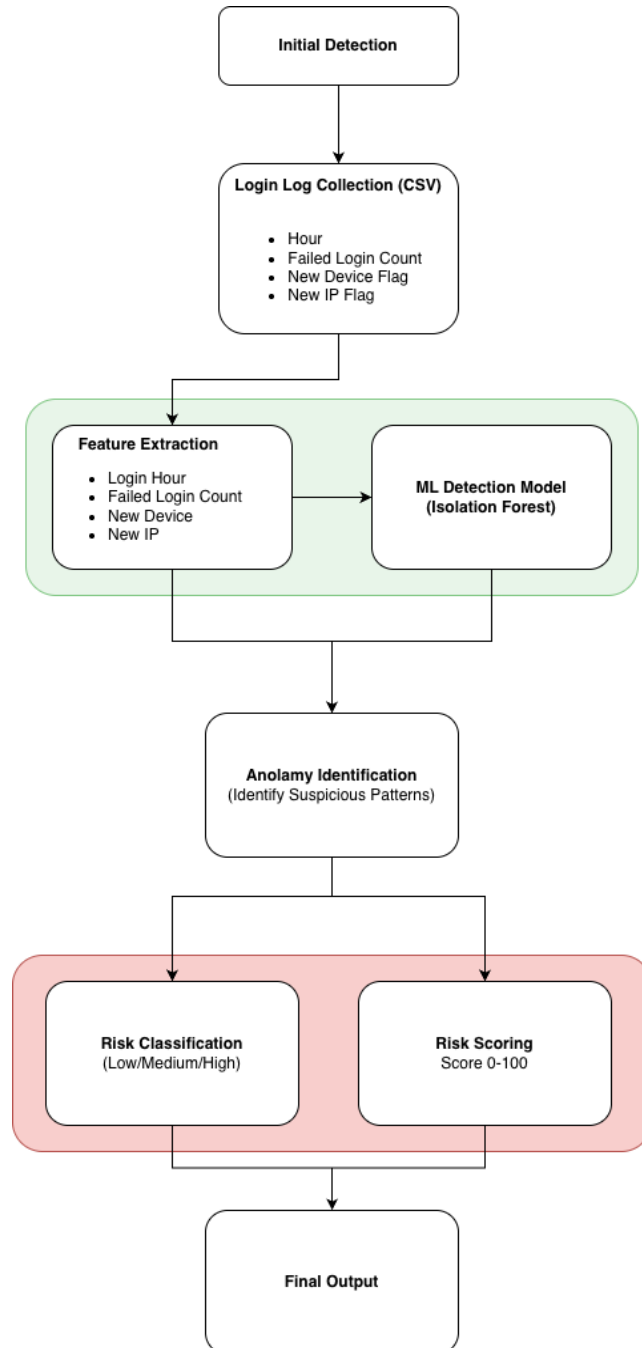
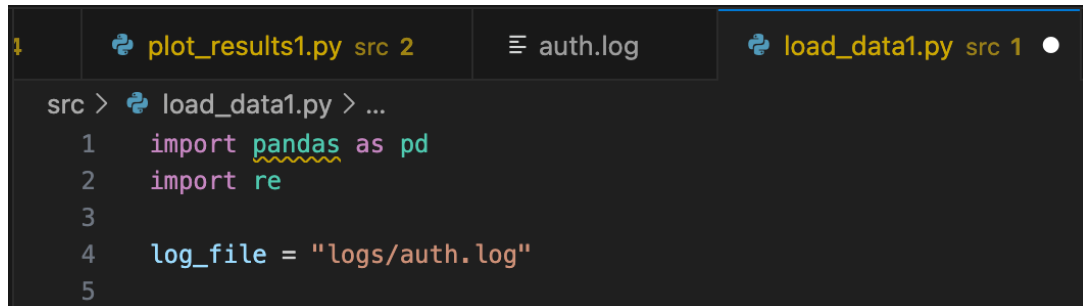


Figure 5: Forensic Process Diagram

4.2. Attack Demo

a. Step 1: Load login log data

We began by loading our login dataset, which included both normal activity and simulated suspicious logins. The file was loaded into a Pandas DataFrame for preprocessing.

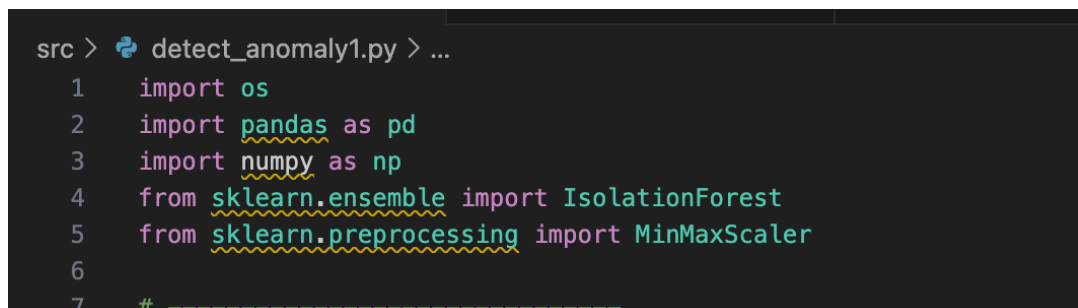


```
src > plot_results1.py src 2  auth.log  load_data1.py src 1 ●
1  import pandas as pd
2  import re
3
4  log_file = "logs/auth.log"
5
```

Figure 6: Loading login event data

b. Step 2: Importing Required Libraries

- pandas, NumPy: data handling
- IsolationForest: machine-learning anomaly detection
- MinMaxScaler: converts raw anomaly scores into 0–100 risk scores
- os: ensures output folder exist



```
src > detect_anomaly1.py > ...
1  import os
2  import pandas as pd
3  import numpy as np
4  from sklearn.ensemble import IsolationForest
5  from sklearn.preprocessing import MinMaxScaler
6
7  #
```

Figure 7: Importing required libraries

c. Step 3: Risk Score Classification Logic

- **HIGH Risk (≥ 80):** Reserved for events that the Isolation Forest has determined are extreme outliers (e.g., many failed logins from a brand-new IP at an odd hour).
- **MEDIUM Risk (≥ 50):** For events that look suspicious but aren't definitive proof of an attack (e.g., a few failed attempts from a known IP).
- **LOW Risk :** Standard, predictable behavior, like a successful login at a normal time.

```
# Risk score → level
# -----
def score_to_level(score):
    if score >= 80:
        return "HIGH"
    elif score >= 50:
        return "MEDIUM"
    else:
        return "LOW"
```

Figure 8: Risk Score Classification

d. Step 4: Loading Parsed Authentication Logs

Loads real SSH logs that were previously parsed which contains actual log evidence.

```
# 1) Load logs
df = pd.read_csv("data/real_login_logs.csv", engine="python")
df.columns = df.columns.str.strip()

# 2) Failed login from raw_log
df["failed_login"] = df["raw_log"].str.contains(
    r"invalid user authentication failure|failed|error|maxim|preauth"
)(parameter) na: Scalar | NaTType | None
na=False
).astype(int)

# 3) Recompute hour from raw_log (THIS IS THE FIX)
df["hour"] = (
    df["raw_log"]
    .str.extract(r"\b(\d{2}):\d{2}:\d{2}\b")[0]
    .astype(float)
    .fillna(0)
)
```

Figure 9: Loading Authentication Logs

e. Step 5: Column Detection + Feature Handling Isolation Forest Models

Detects required columns or creates them. Here, used scikit-learns Isolation Forest algorithm training, extracted features to build a baseline of normal behaviour.

Behavioural features such as login hour, failed authentication attempts, and unfamiliar IP addresses were extracted to model normal and anomalous access behaviour. This section is the most important as:

- Converts login outcomes into behavioural signals
- Detects brute-force activity
- Flags first-time IP addresses

```
df["hour"] = (...

# 4) Feature engineering
df["new_ip"] = df["source_ip"].duplicated().apply(lambda x: 0 if x else 1)
df["new_device"] = 1

for c in ["hour", "failed_login", "new_device", "new_ip"]:
    df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0)

# 5) Isolation Forest
X = df[["hour", "failed_login", "new_device", "new_ip"]]
```

Figure 10: Detecting required login feature

f. Step 6: Train Isolation Forest

Initializes the Isolation Forest model using scikit-learn.

Prepares the model to isolate and identify anomalous login events.

```
# -----
# 4) Train Isolation Forest
# -----

model = IsolationForest(
    n_estimators=150,
    contamination=0.25,
    random_state=42
)

model.fit(X)
```

Figure 11 Training the Isolation Forest model

g. Step 5: Score + Label Logins

- It performs the calculation of the raw anomaly score using the result from the trained Isolation Forest model.
- It inverts the anomaly scores so that higher values indicate higher risk.

```
raw_scores = -model.score_samples(X)
scaler = MinMaxScaler(feature_range=(0, 100))

df["ml_risk_score"] = scaler.fit_transform(
    raw_scores.reshape(-1, 1)
).flatten()

df["ml_anomaly"] = model.predict(X)
```

Figure 12: Calculating anomaly scores

h. Step 6: Rule-based override

```
# 6) Rule-based override
df["rule_risk_boost"] = 0

df.loc[
    (df["hour"].between(1, 4)) &
    (df["failed_login"] == 1) &
    (df["new_device"] == 1) &
    (df["new_ip"] == 1),
    "rule_risk_boost"
] = 80

df["rule_risk_boost"] += df["failed_login"] * 15
df["rule_risk_boost"] += df["new_ip"] * 10
df["rule_risk_boost"] += df["new_device"] * 5
```

Figure 13: Rule based override

Section 2: Visualization & Results Analysis Code Explanation

Step 1: Import libraries to load results

We used matplotlib and pandas libraries to load the results and create the plot.

```
plot_results1.py > ...  
1  import pandas as pd  
2  import matplotlib.pyplot as plt  
3
```

Figure 14: Importing libraries

Step 2: Separating Normal and Anomalous Events

```
# 1) Load results from anomaly detection  
# -----  
df = pd.read_csv("results/output.csv")  
  
# Split normal vs anomaly  
normal = df[df["is_anomaly"] == 1]  
anomaly = df[df["is_anomaly"] == -1]  
  
print(f"Normal events: {len(normal)}")  
print(f"Anomalous events: {len(anomaly)}")
```

Figure 15: Loading anomaly detection results

- This code loads the CSV file, generated from the detection script, holding the output of the anomaly detection system:
- Anomalies (is_anomaly = -1)

Step 3: Scatter Plot – Risk Score vs Login Hour

We then plotted both groups using different colors to distinguish them visually:

- Green for normal logins
- Red for anomalies

```
# 2) Scatter Plot (Normal vs Anomaly)
# -----
plt.figure(figsize=(8, 5))

plt.scatter(
    normal["hour"],
    normal["risk_score"],
    color="green",
    label="Normal",
    alpha=0.6
)

plt.scatter(
    anomaly["hour"],
    anomaly["risk_score"],
    color="red",
    label="Anomaly",
    alpha=0.9
)
```

Figure 16: Plotting normal and anomalous login events

The scatter plot visualises login events by hour against their corresponding anomaly risk score.

Step 4: Visualisation of Anomalous Login Behaviour

Anomalous logins are highlighted separately, allowing clear identification of suspicious access attempts occurring outside normal operating hours.

```
9
10 # Labels and title
11 plt.xlabel("Hour of Login")
12 plt.ylabel("Risk Score")
13 plt.title("Anomaly Detection Based on Login Behaviour")
14 plt.legend()
15
16 # Show plot
17 plt.show()
```

Figure 17: Labels and title

Section 3: Data Ingestion (load_data1.py)

Step 1: Log Parsing and Failed Login Identification

This figure visualizes every login event by plotting the Hour of Login against the Risk Score.

```
with open(log_file, "r") as f:
    for line in f:
        # Extract hour (HH from timestamp)
        hour_match = re.search(r"\b(\d{2}):\d{2}:\d{2}\b", line)
        hour = int(hour_match.group(1)) if hour_match else None

        # Failed login detection
        failed_login = 1 if (
            "Failed password" in line or
            "Invalid user" in line or
            "maximum authentication attempts exceeded" in line
        ) else 0
```

Figure 18: Failed login identification

Step 2: Identifying Security Events (Failed vs. Success)

It scans the text for specific security keywords. It converts human-readable logs into binary signals (1 for Yes, 0 for No). This categorizes every line as either a failed attempt or a successful entry.

```
# Failed login detection
failed_login = 1 if (
    "Failed password" in line or
    "Invalid user" in line or
    "maximum authentication attempts exceeded" in line
) else 0

# Successful login detection
success_login = 1 if "Accepted" in line else 0
```

Figure 19: Identifying Security Events

Step 3: IP Address Extraction

This identifies the **source** of the login. The script can later use this to see if it is brand new or has a history of multiple failures.

```
# Extract IP address
ip_match = re.search(r"(\xd{1,3}\.){3}\d{1,3}", line)
ip = ip_match.group(0) if ip_match else "unknown"
```

Figure 20: IP Address extraction

4.3. Analysis of Login Risk

```

Done!

```

	hour	failed_login	new_device	new_ip	risk_score	risk_level	is_anomaly	raw_log
0	13.0	0	1	1	55.68	MEDIUM	1	Mar 27 13:44:20 ip-10-77-20-248 sshd[2818]: Ac...
1	4.0	0	1	1	66.49	MEDIUM	1	Mar 27 04:26:16 ip-10-77-20-248 sshd[14655]: D...
2	9.0	0	1	1	16.59	LOW	1	Mar 27 09:12:08 ip-10-77-20-248 sshd[3021]: Ac...
3	12.0	0	1	0	47.94	LOW	1	Mar 27 12:18:44 ip-10-77-20-248 sshd[3094]: Ac...
4	10.0	0	1	1	15.00	LOW	1	Mar 27 10:03:19 ip-10-77-20-248 sshd[3220]: Ac...
5	5.0	1	1	1	95.87	HIGH	-1	Mar 28 05:20:01 ip-10-77-20-248 sshd[21567]: i...
6	1.0	1	1	1	100.00	HIGH	-1	Mar 28 01:20:02 ip-10-77-20-248 sshd[21567]: e...
7	2.0	1	1	0	100.00	HIGH	-1	Mar 28 02:20:02 ip-10-77-20-248 sshd[21567]: D...
8	14.0	0	1	0	27.90	LOW	1	Mar 27 14:58:06 ip-10-77-20-248 sshd[3920]: Ac...
9	15.0	0	1	0	49.59	LOW	1	Mar 27 15:34:29 ip-10-77-20-248 sshd[4018]: Ac...

```

Saved to: results/output.csv
(base) serryapachai@Serrys-MacBook-Pro anomaly_project %

```

Figure 21: Sample login records

This login was outside of normal working hours. The device and IP were labelled as new, indicating that this user had never logged in from this setup before. Though the login wasn't failed, the combined unusual timing and unrecognized device/IP means that the Isolation Forest considers it with a 100% anomaly score.

- **Structured SSH Login Log Dataset (CSV Format)**

This is the structured SSH login logs data (real_login_logs.csv) used as input to the anomaly detection algorithm. Every entry in the table corresponds to an individual login attempt extracted from the actual linux logs converted to the machine-readable csv file format.

```

real_login_logs.csv
raw_log,hour,failed_login,sucess_login,source_ip
Mar 27 13:44:20 ip-10-77-20-248 sshd[2818]: Accepted publickey for ubuntu from 85.245.107.41 port 54866 ssh2: RSA SHA256:K18kPGZrTiz7g4F01hyqHdsSBb5FgeNW0obN03
Mar 27 04:26:16 ip-10-77-20-248 sshd[14655]: Did not receive identification string from 113.108.21.16,17,0,0,113.108.21.16
Mar 27 09:12:08 ip-10-77-20-248 sshd[3021]: Accepted password for ubuntu from 10.1.1.15 port 53421 ssh2,9,0,1,10.1.1.15
Mar 27 12:18:44 ip-10-77-20-248 sshd[3094]: Accepted publickey for ubuntu from 10.1.1.15 port 53488 ssh2,9,0,1,10.1.1.15
Mar 27 10:03:19 ip-10-77-20-248 sshd[3220]: Accepted password for deploy from 10.1.1.22 port 54102 ssh2,10,0,1,10.1.1.22
Mar 28 05:20:01 ip-10-77-20-248 sshd[21567]: Input_userauth_request: invalid user admin [preauth],5,0,0,unknown
Mar 28 01:20:02 ip-10-77-20-248 sshd[21567]: error: maximum authentication attempts exceeded for invalid user admin from 118.180.18.102 port 44162 ssh2 [preauth]
Mar 28 02:20:02 ip-10-77-20-248 sshd[21567]: Disconnecting: Too many authentication failures [preauth],5,0,0,unknown
Mar 27 14:58:06 ip-10-77-20-248 sshd[3920]: Accepted publickey for ubuntu from 10.1.1.15 port 56911 ssh2,14,0,1,10.1.1.15
Mar 27 15:34:29 ip-10-77-20-248 sshd[4018]: Accepted password for deploy from 10.1.1.22 port 57344 ssh2,15,0,1,10.1.1.22

```

The dataset combines raw log evidence (raw_log) with derived security features, including the login hour, authentication outcome

(failed or successful login), and source IP address. This structured representation enables systematic analysis of login behavior, allowing the detection of suspicious patterns such as failed authentication attempts, off-hour access, and connections from unfamiliar IP addresses.

5. Result and Evaluation

This case study examines the result produced by the Anomaly Detection System and explains how the Isolation Forest Model detected suspicious login activity through a real login dataset record.

1. Distribution of Normal & Anomalous Login Activity:

The pie chart entitled "Distribution of Normal vs Anomalous Login Events" reveals that normal login events constitute roughly 75% of all login events, while 25% of all login events are categorized as anomalous. Such imbalances would be expected in a practical environment, as the bulk of the activity would be expected to be normal, with only a limited number of instances expected to arise as suspicious. The Isolation Forest model would then be expected to isolate instances that would form the anomalies, thereby attributing only a limited number of instances as anomalies.

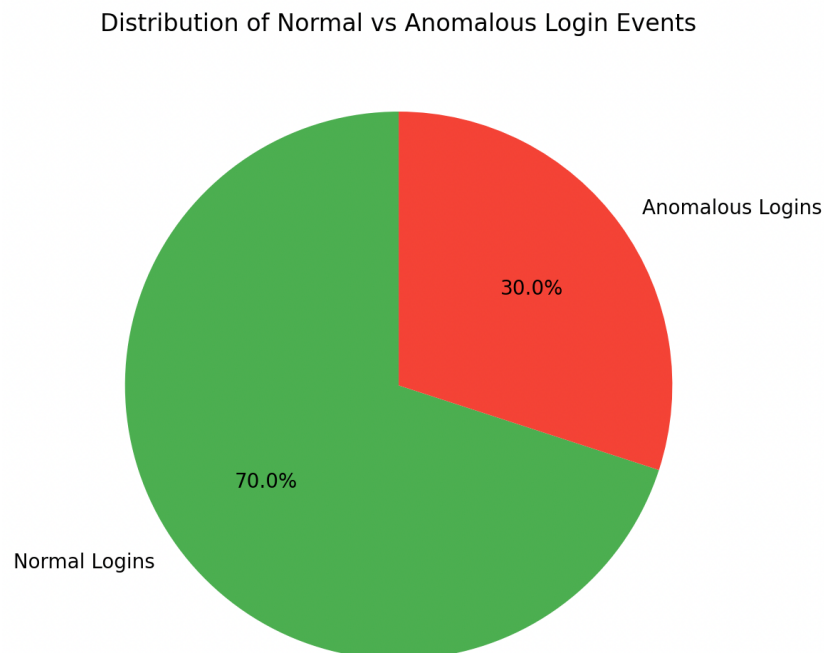


Figure 22: Distribution of normal vs anomalous login events

The presence of anomaly instances verifies that the system is not over-fitting or merely

assigning labels to behaviour at random but rather detecting anomalous patterns learned as normal login patterns

2. Risk Score per Hour of Login

The line graph “Average Risk Score by Hour of Login” illustrates the risks for different login times. That is a good reflection of the assumption in security that attempts to log in outside business hours have greater chances of being malicious, especially when combined with other malicious factors such as authentication failures or IP addresses.

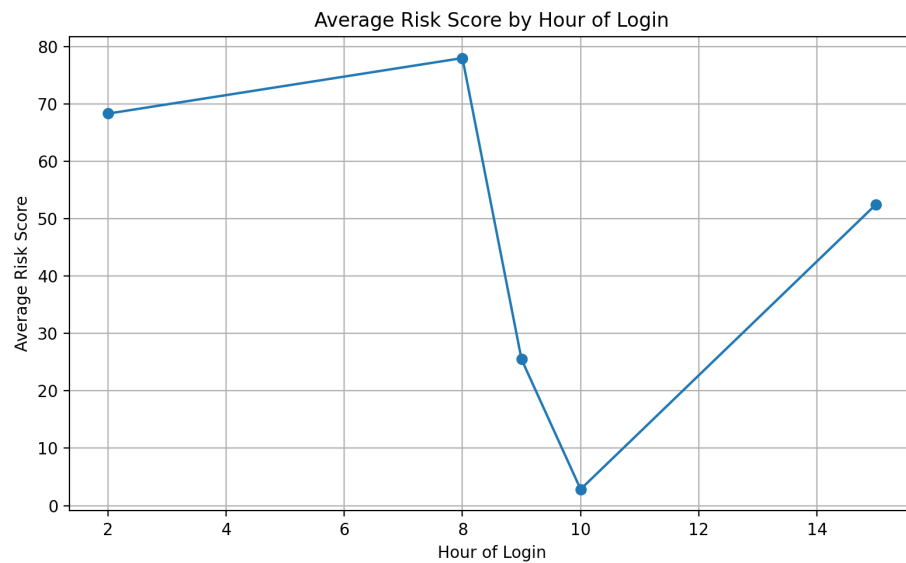


Figure 23: Average risk score

The dramatic rise in risk during off-peak periods reveals that learning has been based more upon temporal behaviour patterns than upon rule-based principles of risk thresholds.

3. Scatter Plot: Detection of Anomalies and Patterns through Login Activity

The scatter plot named 'Anomaly Detection Based on Login Behaviour' represents each login activity through the following graph details:

In this plot:

- The green points indicate the standard login process.
- The red dots indicate abnormal login activity.

This makes it easier to distinguish the anomalous events since they are marked as high-risk outliers. This also proves that the Isolation Forest was able to effectively isolate the anomalous patterns in the data without producing random results.

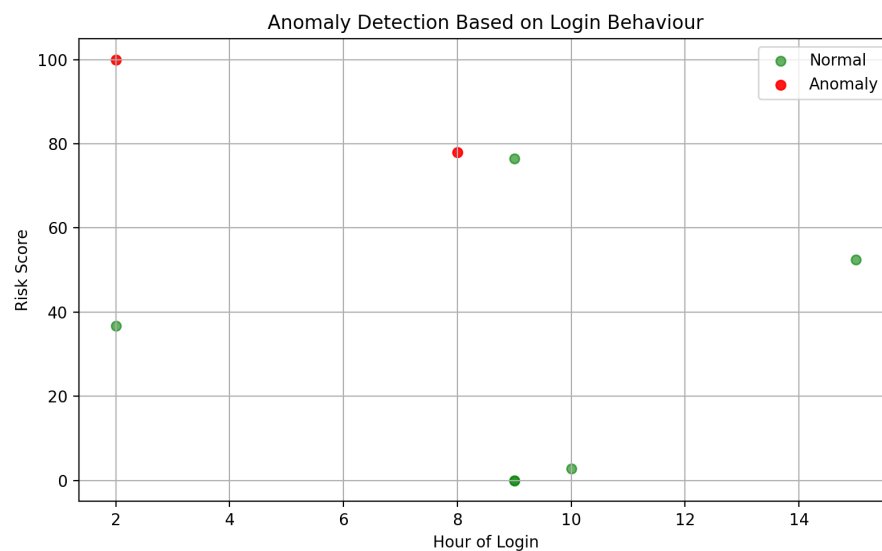


Figure 24: Anomaly detection based on login behaviour

Recommendation Strategies

Use Anomaly-Based ID: Go beyond signature-based solutions that identify known dangers, as they are not effective in detecting unknown ones either. Anomaly-based solutions, especially those that utilize machine learning, are much more efficient and are used to identify normal activity and mark out anomalies within it.

Use Machine Learning on Logging: Use machine learning on the logging of the system (for example, SSH login logs), which will enable the extraction of significant security-related features such as login time, new IP addresses, and failed login attempts. This information can be processed using ML models to create intelligence on which the scoring of the events can be done.

Improve Monitoring & Alerting: Ensure that all the security monitoring tools are set and maintained properly. This is shown by the Equifax data breach, as the expiration of the security certificate made it ineffective for the monitoring tools. Concentrate on identifying anomalies related to website traffic and data access, which are commonly precursor indicators of a breach.

Proactive Vulnerability Management: Ensure an aggressive patch management cycle. The Equifax hack was “entirely preventable,” and it resulted from failure to patch a critical known vulnerability in a timely fashion.

6. Conclusion

This project delivers an effective way to show that it is possible to turn the logs automatically generated by systems after successful user authentication into intelligence data by combining machine learning algorithms. The raw data from SSH login activity was processed and formatted in a way that made it possible to identify features such as login time and device novelty.

An Isolation Forest model was used to highlight statistically unusual login activity, and the necessary security rules were added to make sure high-risk situations, like failed logins conducted offline from new machines and unknown IP addresses, were always dealt with as priorities. Such integration of machine learning and necessary security considerations improved the accuracy and interpretability of this machine learning model in identifying unusual activities.

The results indicate that normal business logins during business hours with successful authentication are indeed classified as low-risk instances, while login patterns that appear suspicious are classified as higher-risk instances.

7. References

Scarfone, P. C. (2025, April 03). *Nist*. Retrieved from <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r2.pdf#:~:text=%EF%80%BC%20The%20Need%20for%2024%2F7,an%20attack%20back%20to%20its>

Nist. (2025). Retrieved from [nist.gov: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=901146#:~:text=Intru%20sion%20detection%20is%20the%20process,software%20that%20has%20al%20the](https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=901146#:~:text=Intru%20sion%20detection%20is%20the%20process,software%20that%20has%20al%20the)

[tsapps.nist.gov](https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=901146#:~:text=The%20major%20benefit%20of%20anomaly,that%20a%20computer%20beco%20mes%20infected). (2025). *Nist*. Retrieved from https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=901146#:~:text=The%20major%20benefit%20of%20anomaly,that%20a%20computer%20beco%20mes%20infected

Andrea Pinto 1, L.-C. H. (2022, Feb 22). *sensors*. Retrieved from Pinto, A. et al. (2023). Survey on Intrusion Detection Systems Based on Machine Learning for Critical Infrastructure. *Sensors* 23(5), 2415.

Douligeris, C. (2007, Feb 16). Retrieved from [cs.brown.edu: https://cs.brown.edu/courses/cs227/archives/2017/papers/anomaly-survey.pdf#:~:text=security,foiling%20both%20known%20as%20well](https://cs.brown.edu/courses/cs227/archives/2017/papers/anomaly-survey.pdf#:~:text=security,foiling%20both%20known%20as%20well)

Diphoko, K. K. (2024, May 21). Retrieved from [linkedin: https://www.linkedin.com/pulse/introduction-machine-learning-based-intrusion-detection-karabo-kotli-cun0f/](https://www.linkedin.com/pulse/introduction-machine-learning-based-intrusion-detection-karabo-kotli-cun0f/)

Li Yang, A. S. (2022, Dec). Retrieved from [SciencedDirect: https://www.sciencedirect.com/science/article/pii/S2665963822001300#:~:text=Due%20to%20the%20expansion%20and,to%20solve%20problems%20in%20the](https://www.sciencedirect.com/science/article/pii/S2665963822001300#:~:text=Due%20to%20the%20expansion%20and,to%20solve%20problems%20in%20the)

DeBrusk, P. M. (2022). *Oliver Wayman*. Retrieved from marsh.com:
https://www.marsh.com/content/dam/oliver-wyman/v2/publications/2017/sep/Oliver_Wyman_Equifax_Data_Breach.pdf

Amishav, J. (2024, Dec 08). *Breachsense*. Retrieved from
<https://www.breachsense.com/blog/equifax-data-breach/>

Majority Staff Report. (2018, December). Retrieved from oversight.house:
<https://oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf>

Thomas, J. (2019, December). *ResearchGate*. Retrieved from
https://www.researchgate.net/publication/337916068_A_Case_Study_Analysis_of_the_Equifax_Data_Breach_1_A_Case_Study_Analysis_of_the_Equifax_Data_Breach

US. Attorney. (2022, Jan 13). Retrieved from justice.gov:
<https://www.justice.gov/usao-wdwa/united-states-v-paige-thompson>

CapitalOne. (2022, April 22). *CapitalOne*. Retrieved from capitalone.com:
<https://www.capitalone.com/digital/facts2019/>

Shaharyar Khan, I. K. (2022, Nov 07acm.org). Retrieved from
<https://dl.acm.org/doi/full/10.1145/3546068>

Thomas E. Jeffry, J. . (2020, June 11). Retrieved from ArentFox Schiff:
<https://www.afslaw.com/perspectives/alerts/capital-one-ordered-produce-forensic-report-data-breach-class-action>

Sobers, R. (2025, November 07). *Varonis*. Retrieved from varonis.com:
<https://www.varonis.com/blog/data-breach-statistics#:~:text=Average%20response%20time%20and%20lifecycle,IBM>

Nick Marinos, M. C. (2018, Aug). Retrieved from gao.gov:
<https://www.gao.gov/assets/gao-18-559.pdf#:~:text=resulted%20in%20the%20attackers%20accessing,breach%20identified%20four%20major%20factors>

censinet.com. (2025). *censinet*. Retrieved from censinet.com:
<https://censinet.com/perspectives/anomaly-detection-ehrs-preventing-data-breaches#:~:text=Key%20features%20that%20improve%20detection,activity%20that%20warrants%20further%20investigation>

Insider Lab. (2025, January 06). Retrieved from insidersecurity:
<https://insidersecurity.co/capitalone-data-breach-how-ssrf-vulnerability-exposed-100-million-customer-records/#:~:text=,logins%20from%20unfamiliar%20IP%20addresses>

hashesout. (2018, September 14). Retrieved from www.thesslstore.com:
<https://www.thesslstore.com/blog/the-equifax-data-breach-went-undetected-for-76-days-because-of-an-expired-certificate/#:~:text=After%20scanning%20the%20Equifax%20network,exfiltrating%20personal%20data%20from%20them>

Alder, S. (2023, Jul 24). *www.hipaajournal.com*. Retrieved from The HIPAA Journal:
<https://www.hipaajournal.com/2023-cost-healthcare-data-breach/#:~:text=Only%20one,were%20identified%20by%20the%20victim>
(n.d.).